

Embedded Systems Architectures - Laboratory Guide 7

Low power consumption modes

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Familiarization with the ESP32-C6 operating modes
- Use of light-sleep and deep-sleep power consumption modes
- Use of wake-up modes via the RTC timer, GPIOs, and EXT1 signal

2. Introduction

The ESP32-C6, like other microcontrollers in the Espressif ESP family, integrates a wide range of power management modes designed to optimize the balance between computational performance and energy efficiency. This functionality is particularly relevant in battery-powered applications, IoT (Internet of Things) devices, environmental monitoring systems, remote sensors, and portable equipment, where autonomy and reduced consumption are critical factors.

The chip was designed with a low-power architecture based on a 32-bit RISC-V core, complemented by various elements (e.g., communication peripherals) and a dynamic power regulation system that allows selectively switching off internal blocks when they are not in use. In this way, the ESP32-C6 can continuously adapt to application requirements, switching between full operation modes and deep-sleep modes with consumptions on the order of a few microamperes. This flexibility allows the microcontroller to operate in different power modes, such as Active, Modem-sleep, Light-sleep, and Deep-sleep, adjusting the activity level of the internal components (CPU, Wi-Fi radio, Bluetooth LE, Real-Time Clock, memory, and

peripherals). The result is a versatile platform that can, for example, operate in active mode for high-speed Wi-Fi transmissions and then enter a "deep sleep" state with less than 10 μA of current consumption, preserving essential data in the Real-Time Clock (RTC) memory. This capability makes the ESP32-C6 especially suitable for data collection devices, distributed control systems, telemetry modules, and long-lasting industrial applications where it is necessary to combine advanced connectivity with maximum energy efficiency.

3. Low Power Modes

The ESP32-C6 supports 5 low-power modes that give developers great flexibility in managing its operation, offering a wide range of trade-offs between performance and energy optimization.

- **Active mode** (Full power): all peripherals and interfaces, such as SPI, I2C, Bluetooth, Wi-Fi and both CPU cores, are active. This mode guarantees maximum performance, but also the highest consumption, which can reach 260 mA during peak Wi-Fi transmission and about 120 mA in continuous transmission.
- **Modem-sleep**: the main processor is still active, but the radio module (Wi-Fi/Bluetooth) goes into sleep mode. It allows significant energy savings, with consumption ranging from 3 mA to 20 mA, depending on the processor frequency and the tasks being executed. This mode is useful when the device processes data locally without continuous network communication.
- **Light-sleep**: the CPU is paused, maintaining the contents of RAM and the internal state of the main peripherals. Wireless communication interfaces are turned off and the power to the digital sections is reduced. Power consumption drops to around 0.8 mA, with fast wake-up times (microseconds). It is suitable for short periods of inactivity where it is important to preserve the application context.
- **Deep-sleep**: the CPU and almost all peripherals are turned off. Only the RTC (Real-Time Clock) and, optionally, the ULP (Ultra-Low Power) coprocessor

remain active. This coprocessor, which can be programmed in assembly, allows simple tasks such as reading sensors via I2C or ADC and waking the system when a value exceeds a limit. Power consumption ranges from 150 μ A (with ULP active) to 10-25 μ A and can drop to 2.5 μ A in hibernation mode, when only the RTC clock remains active.

- **Hibernation:** operating mode that offers the lowest power consumption; All digital logic and memory are switched off, leaving only the RTC, which allows the system to wake up at long, predefined intervals. In battery-powered applications, this mode can guarantee more than a year of autonomy, depending on the wake-up frequency.

The ESP32-C6 integrates a versatile set of wake-up sources that enable the system to be reactivated from various sleep modes efficiently and flexibly. These sources are fundamental to optimize the balance between energy autonomy and responsiveness, allowing the microcontroller to remain inactive for long periods, waking up only when necessary to perform specific tasks.

The wakeup sources can be timed, external, or internal and are managed by the Real-Time Clock (RTC) controller, which remains active even when the chip's other elements are switched off. Among the main wakeup options, the following stand out:

- **RTC Timer:** Allows the chip to wake up after a predefined time interval, ideal for periodic applications such as data collection or sending measurements.
- **External GPIO (EXT1):** Enables the system to wake up when one or more digital pins change their logic state (high or low level), functioning as a hardware interrupt.
- **ULP Coprocessor:** Can perform simple tasks while the chip is in deep sleep, such as reading I2C sensors or ADC values, and wake up the system when a condition is met.
- **Touch Sensor:** In compatible versions, allows touching a capacitive pin to enable the chip, useful in low-power human interfaces.

- **UART** (Light-sleep only): The chip can wake up when it detects activity on an RX serial line, making it suitable for GPS modules, modems, or serial sensors.

The choice of wake-up source depends on the sleep mode used and the app's requirements. In light-sleep mode, waking up is almost immediate and the system state is preserved; in deep-sleep mode, the chip performs a controlled reset, but can retrieve data previously stored in the RTC memory.

In this lab assignment, we will explore the use of light-sleep and deep-sleep modes.

4. Practical Work

The practical work in this class is based on the Reference API for the ESP32-C6 sleep modes and the ESP32-C6 Technical Reference Manual: https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf

4.1 Active Mode

In active mode, all peripherals and interfaces are active, ensuring the fastest response but also resulting in the highest power consumption.

Important note! In this section, the objective is only to become familiar with the power consumption associated with the operation of the circuit from the lab assignment “SPI Interface and SD Card” operating in active mode, as shown in the provided graphs. It is not necessary to assemble the circuit or program the ESP32-C6.

Figures 1 and 2 illustrate the current consumption measured with the Nordic Power Profiler Kit II for the application developed in that lab assignment to read data from the DHT20 sensor and write it to the microSD card (the corresponding circuit is shown in Figure 3).



Figure 1 – Current consumption in normal mode.

As shown in Figure 1, the average consumption in "standby" mode is 34.49 mA, with peaks up to 40.94 mA, while during the read/write period, it is 42.16 mA, with peaks up to 72.16 mA (Figure 2). It is also easy to see that the sampling period for these variables is consistent with the specifications of the respective lab assignment: 1 second between successive samples.

Although the peripherals that consume the most energy (Wi-Fi, Bluetooth, IEEE 802.15.4) are not yet active, the standby power consumption in this application is already considerable and may be incompatible with applications with high autonomy requirements without a permanent power source. The next section addresses this problem by presenting the light-sleep operating mode.



Figure 2 – Current consumption for reading the humidity and temperature and writing to the micro SD card.

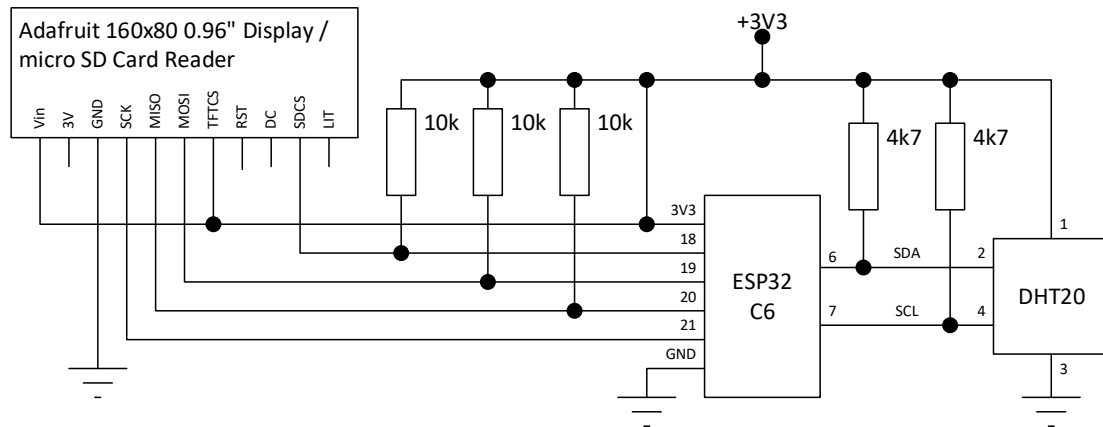


Figure 3 – Circuit for integrating the card reader with the DHT20 sensor.

4.2 Light Sleep Mode

The light-sleep operating mode benefits from a dramatic reduction in microcontroller power consumption and very short wake-up times, in the microsecond range, enhancing its use in applications where response time is critical.

Important warning regarding the programming interface and low-power modes! In the various examples to be studied on the use of low-power modes, program the microcontroller via the UART on the alternative USB-C plug (typically named /dev/ttyUSB0). When the ESP32-C6 enters deep-sleep or light-sleep modes, the SoC disables or shuts down the subsystems on which the USB/JTAG interface depends, integrated into the board, thereby disconnecting the chip from the computer. Since the alternative UART port is enabled via an external UART-USB converter that does not depend on the SoC's power supply, entering deep sleep or light sleep modes has no effect on this programming interface.

Important warning regarding the current consumption plots! The graphs presented were obtained by a device (Nordic Power Profiler Kit II) that is not available to students. These graphs serve only as references for use in the various operating modes.

4.2.1 Using the timer as a wake-up source

In this part, the objective is to put the microcontroller into light-sleep mode and wake it up periodically using the RTC (Real-Time Clock) controller timer. To do this, download the file *ls_wktimer.zip* available on the course webpage and extract it to your working directory.

This program configures the RTC's internal timer to generate a wake-up event. During light-sleep mode, the CPU, most peripherals and the main oscillator will be suspended. The RTC (Real-Time Clock) domain remains active with its slow oscillator, which the RTC timer uses to keep track of time and generate a signal that wakes up the processor when the defined period ends.

Question 1: Open the project in VSCode and study the *main.c* file. What is the duration of the configured light-sleep period?

Question 2: Proceed with program compilation, download it to the ESP32-C6 SoC, and then open the corresponding console. You should see an output in the terminal similar to the following:

```
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
going for a nap
napped for 999
going for a nap
napped for 999
going for a nap
napped for 999
```

Question 3: Modify the program by configuring different time intervals between wake-ups and check its operation.

Using the original program tested in Question 2 and the Nordic Power Profiler Kit II, the current consumption was measured across different phases of operation, as illustrated in Figures 4 and 5. As shown, during the 988 ms period, the average consumption is 239.19 μA , with peaks up to 259.83 μA (Figure 4). In normal mode,

consumption increases substantially to an average of 26.25 mA, with peaks up to 34.39 mA (Figure 5).



Figure 4 – Current consumption in light-sleep mode.

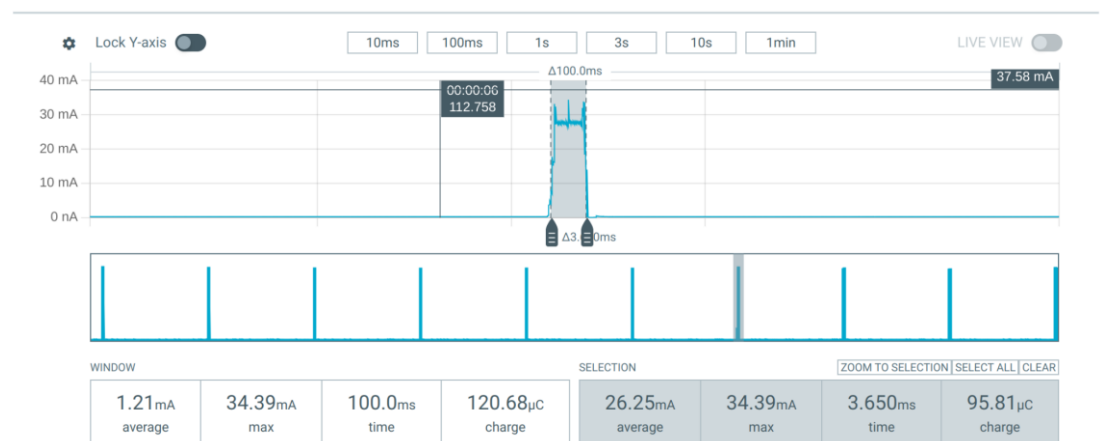


Figure 5 – Current consumption in normal mode.

Question 4: Consult the documentation regarding Sleep Modes and justify the calling of the `console_flush()` function at the end of the `main()` function loop.

4.2.2 Using GPIO as a wake-up source

An alternative wake-up source for light-sleep mode is using GPIOs. In this part, we will use a push button configured as in the lab assignment about basic I/O (connected to GPIO0 and pulled up internally). To do this, download the `ls_wkgpio.zip` file and extract it to your working directory.

Question 5: Open the project in VSCode and study the *main.c* file. Explain the purpose of the following code block:

```
if (gpio_get_level(INPUT_PIN) == 0)
{
    printf("please release button\n");
    console_flush();
    do
    {
        vTaskDelay(pdMS_TO_TICKS(10));
    }
    while (gpio_get_level(INPUT_PIN) == 0);
}
```

Question 6: Proceed with compiling the program, downloading it to the SoC, and then opening the corresponding console. By pressing the push button, you should wake the SoC from light-sleep mode and observe an output similar to the following in the terminal:

```
I (252) main_task: Started on CPU0
I (252) main_task: Calling app_main()
going for a nap
woke up after 2165 ms, reason: button
please release button
going for a nap
woke up after 695 ms, reason: button
please release button
going for a nap
woke up after 144 ms, reason: button
please release button
going for a nap
woke up after 957 ms, reason: button
please release button
going for a nap
```

In light-sleep mode, timer wake-up is internal and consumes less energy, occurring after a defined time interval, while GPIO wake-up is external and reactive, dependent on the level change on a pin, requiring part of the input logic to remain active, which increases consumption.

As presented for wake-up via timer, the current consumption in the different phases was measured during operation with wake-up via GPIO, as documented in Figures 6, 7, and 8. As shown in Figures 6 and 7, consumption does not exceed 181.6 μA in light-sleep mode, with averages of 160.55 and 161.14 μA , respectively. Note also in these figures that the measured time intervals of 687.1 μs and 136.0 μs are very close to those reported on the console (695 μs and 144 μs , respectively). In normal mode, consumption increases to an average value of 21.81 mA with peaks of up to 34.99 mA (shown in Figure 8).



Figure 6 – Current consumption in light-sleep mode (GPIO wake-up) 1.



Figure 7 – Current consumption in light-sleep mode (GPIO wake-up) 2.



Figure 8 – Current consumption in normal mode.

4.3 Deep-sleep mode

The deep-sleep operating mode enables the microcontroller to consume the lowest possible power by completely shutting down the processor, main RAM, and most peripherals, keeping only the RTC (Real-Time Clock) domain active for essential functions such as timekeeping, variable retention, or wake-up event detection. In this state, consumption drops to the order of microamperes to tens of microamperes, making it ideal for battery-powered or remote monitoring applications with long periods of inactivity. The return to active operation implies a complete system restart, with wake-up times higher than those in light sleep, but acceptable in scenarios where the priority is to maximize energy autonomy at the expense of immediate response.

4.3.1 Using the timer as a wake-up source

In this part, you want to put the microcontroller into deep-sleep mode and wake it periodically using the RTC (Real-Time Clock) timer. To do this, download the file *ds_wktimer.zip* and extract it to your working directory.

Question 7: Open the project in VSCode and study the *main.c* file. What is the expected behavior for this program?

Question 8: Proceed with compiling the program, downloading it to the ESP32-C6 SoC, and then opening the corresponding console. You should see an output in the terminal similar to the following, where the counting variable is incremented in each cycle. However, this does not occur. Why?

Hint: Consult the documentation regarding Sleep Modes.

```
I (251) main_task: Started on CPU0
I (251) main_task: Calling app_main()
going to sleep. woken up 0
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x5 (SLEEP_WAKEUP),boot:0x6f (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x40875730,len:0x1744
load:0x4086b910,len:0xe78
load:0x4086e610,len:0x31c8
entry 0x4086b91a
I (24) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (232) coexist: coex firmware version: b0bcc39
I (251) coexist: coexist rom version 5b8dcfa
I (251) main_task: Started on CPU0
I (251) main_task: Calling app_main()
going to sleep. woken up 1
ESP-ROM:esp32c6-20220919
...
I (251) main_task: Started on CPU0
I (251) main_task: Calling app_main()
going to sleep. woken up 2
```

Question 9: Modify the program so that the counting variable is preserved between successive wake-ups and test your program.

Using the previously identified setup, the current consumption in deep-sleep mode was measured for this application, with wake-up via the RTC timer. As shown in Figure 9, consumption in deep-sleep mode averages 8.26 μA , peaking at 10 μA . In

normal mode, the average current is 30.62 mA, reaching a maximum of 50.04 mA (shown in Figure 10).



Figure 9 – Current consumption in deep-sleep mode (wake-up via RTC timer).

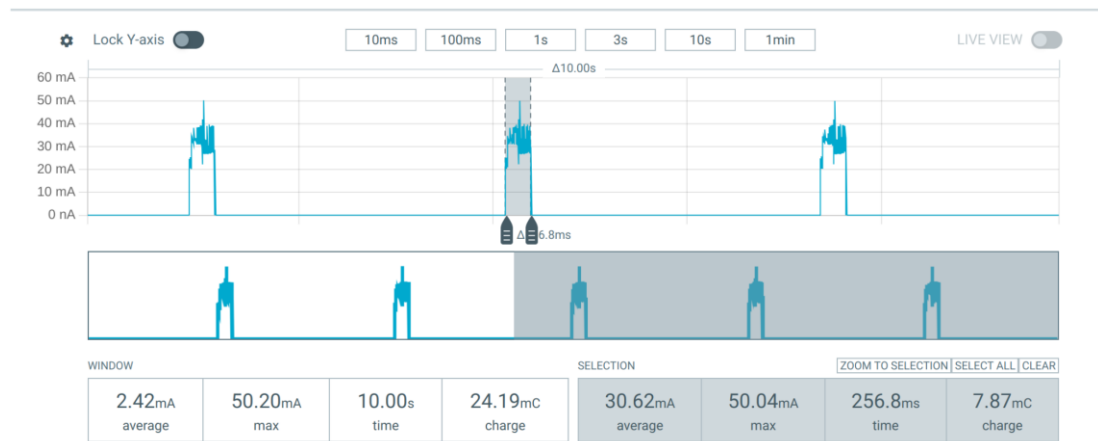


Figure 10 – Current consumption in normal mode.

4.3.2 Using the EXT1 signal as a wake-up source

Awakening from deep-sleep mode via the EXT1 signal allows the ESP32-C6 to respond to an external event detected on one or more input/output pins (GPIOs) configured for this purpose. This mechanism is managed by the RTC controller, which remains partially active during deep sleep and monitors the electrical state of the selected pins, even when the rest of the system is completely off.

When configuring the EXT1 wake-up, you can specify whether the microcontroller wakes up when any pin reaches a certain logic level (ANY_HIGH or ANY_LOW), or only when all pins meet the specified condition. This flexibility allows for the implementation of selective activation strategies, such as waking up the system when a button is pressed, a sensor changes its state, or an external control signal is received.

In this part, we will use the previously assembled push button for wake-up from the light-sleep state (connected to GPIO0). To do this, download the *ds_wkext1.zip* file and extract it to your working directory.

Question 10: Open the project in VSCode and study the *main.c* file. Compile the program and test it on the board by pressing the push button. The result should be the following:

```
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 0
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x5 (SLEEP_WAKEUP),boot:0x6f (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x40875730,len:0x1744
load:0x4086b910,len:0xe78
load:0x4086e610,len:0x31c8
entry 0x4086b91a
I (24) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 1
ESP-ROM:esp32c6-20220919
...
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 2
ESP-ROM:esp32c6-20220919
```

Using the Nordic Power Profiler Kit II, the current consumption was measured across the different phases of operation, yielding the results shown in Figures 11 and 12. As shown, the deep-sleep mode exhibits an average current of 9.64 μA , with a maximum of 11.98 μA (Figure 11). In line with previously measured wake-up via the RTC timer, consumption in normal mode averages 30.79 mA and peaks at 50.36 mA.

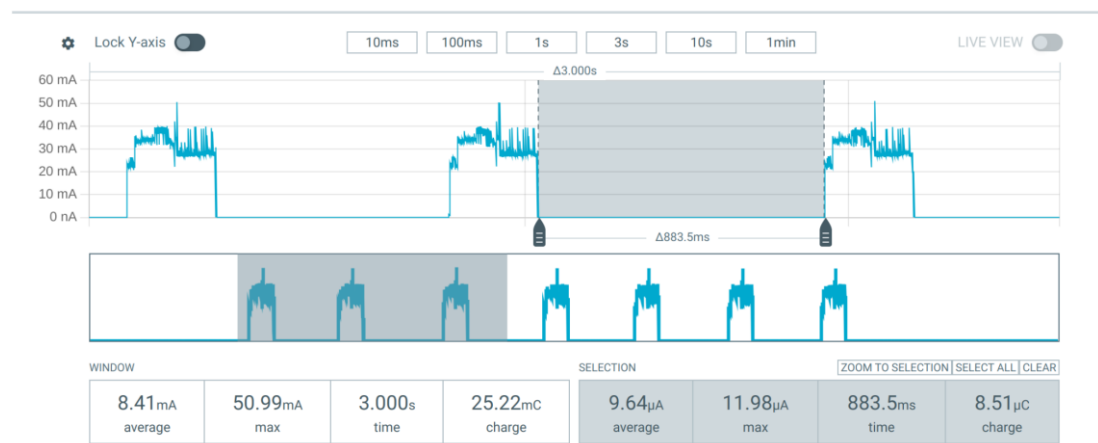


Figure 11 – Current consumption in deep-sleep mode (wake-up via EXT1 signal).

Question 11: Mount a second push button on the pull-up board and modify the program so that when any of the buttons is pressed, the program wakes up from deep-sleep mode.

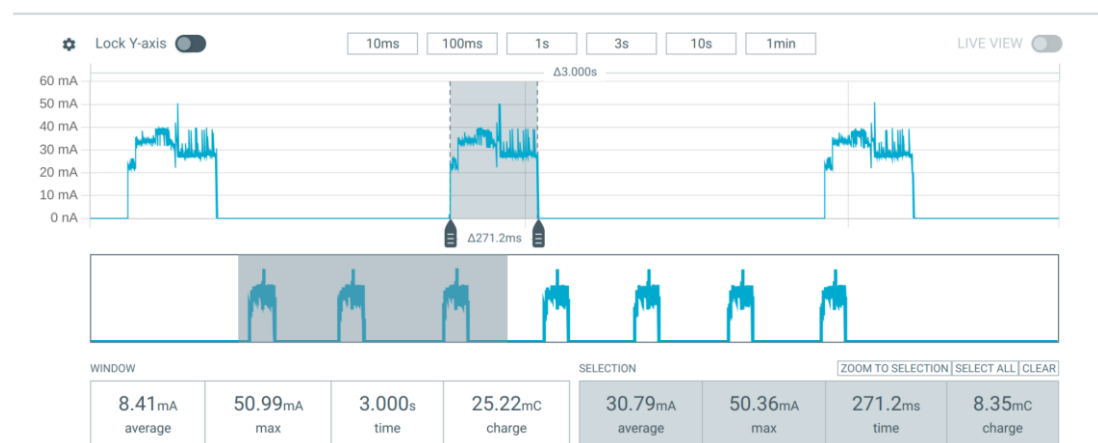


Figure 12 – Current consumption in normal mode.